



SURF

WFO core updates

5.0 → 5.4

Hans Trompert (SURF)

September 22, 2026, Dublin

Summary forms

In your input form generator use:

```
user_input = yield InputForm
user_input_data = extract_user_input(user_input)
summary_data = exclude_summary_fields(user_input_data, {"exclude_from_summary"})

yield from base_summary(product_name, user_input_data)
```

Use `extract_user_input()` to remove form fields with types like `Label` and `Divider`

Use `exclude_summary_fields()` to hide valid input field in the summary

To show before and after field columns in a modify workflow use:

```
yield from base_summary(product_name, new_data=user_input_data, old_data=previous_values)
```



Summary forms

```
summary_options = SummaryOptions(
    product_name=product_name,
    product=TableOptions(
        name="product_summary",
        data=[(extract_user_input(user_input), None)],
    ),
    tables=[
        TableOptions(
            name="item",
            data=make_table_data(new_items, old_items),
            empty_message="No items",
        ),
    ],
)

yield from generate_summary_form(summary_options)
```

- The `base_summary()` is a thin wrapper around `generate_summary_form()`, which accepts a `SummaryOptions`
- Use `product` argument to specify the general product table
- Use `tables[]` to add any number of additional tables with related information
- `make_table_data()` zips a list of new and (optionally) old items into tuples
`TableOptions["data"]` expects



Summary forms

- By default, a field is shown as its field name and `str(value)`
- The frontend translates the field name the same way it translates any other form field
- For fields that need custom rendering, use a `Formatter`
- Formatters are per table, or can be declared global
- A handful of general-purpose formatters ship out of the box:
 - `customer_name_summary_field(get_customer_name_fn)`, returns a formatter that resolves a customer id to its display name via the callable you provide
 - `select_list_summary(field_name)`, returns a formatter that joins a list field into a single, comma-separated row
 - `subscription_summary_fields(subscription_id)`, not a `Formatter` itself, but a helper that yields the standard (subscription_id, description, title) rows for a related subscription



Corelink Modify Summary

Check the settings

When you press submit, the product will be deployed. If something is wrong, correct it now

	Before	After
Customer ID	SURFnet network	SURFnet network
Link member speed	10G	10 Gbit/s
Set maintenance mode for corelink	False	False
Choose node to move to a different node (Optional)		
IS-IS metric	1	10
Overwrite IS-IS metric	False	True

Edit link member 1

Port 1	1/2/9 (IS) (10GBASE-SR) (LEDN005B-JNX-DEV-RG20-2A)	1/2/9 (IS) (10GBASE-SR) (LEDN005B-JNX-DEV-RG20-2A)
Port 2	1/2/9 (IS) (10GBASE-SR) (HB001A-JNX-DEV-RG20-3A)	1/0/2 (PL) (10GBASE-LR) (HB001A-JNX-DEV-RG20-3A)

New link member 2

Port 1	1/0/3 (PL) (10GBASE-SR) (LEDN005B-JNX-DEV-RG20-2A)
Port 2	1/0/7 (PL) (10GBASE-SR) (HB001A-JNX-DEV-RG20-3A)



Callback Step Timeout

```
callback_step(  
    name="Call ext system",  
    action_step=call_ansible_playbook,  
    validate_step=_evaluate_callback_results,  
    timeout=300, # fail if no callback arrives within 5 minutes  
)
```

- `timeout=None` (the default) keeps the original behaviour of waiting forever
- When the deadline passes, the process moves to `FAILED` with `failed_reason = "Callback timed out"`
- Retry re-runs the *entire callback step group*: it executes the `action_step` again and waits anew
- Make sure your `action_step` is idempotent
- Timeouts enforced by `task_validate_awaiting_callbacks`, sweeps every 30 seconds
- to change interval, recreate task with cron schedule at the desired frequency, e.g. `*/15 * * * * *`



oauth2-lib 3.0 uses httpx2 instead of httpx

- The last stable release of the original [httpx](#) (version 0.28.1) occurred in December 2024
- There are no active, unpatched CVEs in [httpx](#) 0.28.x that force an immediate upgrade
- Pydantic, that relies heavily on [httpx](#), created a fork called [httpx2](#) and actively maintains it
- [httpx2](#) is a drop-in replacement, but:
 - Swaps the bundled [certifi](#) package for [truststore](#), using your OS native trust store
 - Replaces `auth=BasicAuth(id, secret)` with `auth=(id, secret)`
 - Import from [httpx2](#) instead of [httpx](#) when using `oauth2-lib`:

```
from httpx2 import AsyncClient, TimeoutException, TransportError
```

- The `orchestrator-core`, and probably other dependencies, still import [httpx](#)
- fortunately, others are actively migrating as well



Schedules API

- The schedules API replaces the now deprecated `@schedule` decorator
- Since 5.0, the GraphQL `scheduledTasks` query no longer returns tasks created by the decorator
- Creation and modification through `POST`, `PUT` and `DELETE` on `/api/schedules`
- Orchestrator-core 5.5 adds a `load_schedules()` helper to load schedules from code
- Very nice UI tab `Metadata/Scheduled task` added to the `orchestrator-ui-library`
- Scheduler CLI:

```
Usage: main.py scheduler [OPTIONS] COMMAND [ARGS]...
```

Commands

<code>run</code>	Starts the scheduler in the foreground.
<code>show-schedule</code>	The <code>`show-schedule`</code> command shows an overview of the scheduled jobs.
<code>force</code>	Force the execution of (a) scheduler(s) based on a schedule ID.
<code>load-initial-schedule</code>	The <code>`load-initial-schedule`</code> command loads the initial schedule using the scheduler API.



Metadata

Products

Product blocks

Resource types

Workflows

Tasks

Scheduled tasks

Edit columns

Export

Schedule task

Task	Task description ↑	Next runtime	Schedule	
 task_clean_up_tasks	Task Clean Up Tasks	16/09/2026, 20:51:14	 interval[6:00:00]	
 task_resume_workflows	Task Resume Workflows	16/09/2026, 16:51:14	 interval[1:00:00]	
 task_validate_awaiting_callbacks	Task Validate Awaiting Callbacks	16/09/2026, 16:03:44	 interval[0:00:30]	
 task_validate_products	Task Validate Products	17/09/2026, 04:30:00	 cron[hour='2', minute='30']	
 task_validate_subscriptions	Task Validate Subscriptions	17/09/2026, 02:10:00	 cron[hour='0', minute='10']	
 task_validate_aggregator_against_subscriptions	Validate aggregator against subscriptions	17/09/2026, 03:50:00	 cron[hour='1', minute='50']	



Create new schedule

Select the task you want to schedule

Validate products

What type of schedule do you need?

Cron

Start date

 Wed Sep 16 2026 15:22:49 GMT+0200

Cron expression *

Cron expression: 5 fields (minute hour day month weekday), or 6 with a leading second. Examples: 0 8 * * 1 (Mondays at 08:00), */15 * * * * * (every 15 seconds)

30 0 1 1 * *

second minute hour day (month) month **day (week)**

- Possible values for this field
- * any value
 - , value list separator
 - range of values
 - / step values
 - 0-6 allowed values
 - SUN-SAT alternative single values
 - 7 sunday (non-standard)

At 01:00:30 AM, on day 1 of the month
Next: 2026-10-01 01:00:30

Reconcile

- Reconcile is now allowed on out of sync subscriptions



The end.

